

wdroid

v2.0.0

Documentação Técnica Completa
CLI Profissional para Gerenciar WhatsApp via Waydroid no Debian

Plataforma-alvo: Debian GNU/Linux 13 (Trixie) — x86_64
Kernel: 6.12.74+deb13+1-amd64 | Waydroid: 1.6.2 | Android: LineageOS 20
Abril · 2026

1. Visão Geral do Projeto

O wdroid é uma CLI modular para gerenciar o ciclo de vida completo do Waydroid — o container Android para Linux — com foco em automatizar a inicialização, diagnóstico, rede, backup e execução do WhatsApp em sistemas Debian.

O projeto surgiu da necessidade de encapsular uma série de comandos manuais repetitivos (waydroid session start, correções de iptables, ADB etc.) em uma única ferramenta coesa, resistente a falhas e observável.

1.1 Objetivos

- Abstrair completamente a complexidade do Waydroid em comandos simples
- Garantir idempotência: executar wdroid start múltiplas vezes não quebra o estado
- Logging estruturado com arquivo diário para rastreabilidade
- Auto-diagnóstico com health score e auto-correção
- Backup seguro com parada automática do container
- Sistema de plugins para extensibilidade sem modificar o core

1.2 Estrutura do Repositório

```
wdroid-v2/
├── bin/
│   └── wdroid                # Entrypoint principal – dispatcher de comandos
├── core/
│   ├── config.sh            # Variáveis globais e timeouts
│   ├── logger.sh            # Logging colorido + arquivo diário
│   ├── lock.sh              # Lockfile anti-concorrência com detecção de órfão
│   ├── utils.sh             # run, retry, wait_for, require_cmd, confirm
│   ├── state.sh             # State machine: STOPPED → APP_RUNNING
│   └── plugin.sh            # Sistema de plugins dinâmico
├── modules/
│   ├── container.sh         # Controle do systemd waydroid-container
│   ├── session.sh           # Controle da sessão Android
│   ├── network.sh           # Diagnóstico e correção de rede
│   ├── app.sh               # ADB, install APK, screenshot, send-text
│   └── backup.sh            # Backup e restauração com parada automática
├── commands/
│   ├── start.sh             # Inicialização por state machine
│   ├── stop.sh              # Encerramento na ordem inversa
│   ├── status.sh            # Status completo e colorido
│   ├── doctor.sh            # Health check com score e --fix
│   ├── reset.sh             # Reset com backup obrigatório
│   └── logs.sh              # Visualização de logs (container/wdroid/all/follow)
├── plugins/
│   └── whatsapp.sh          # Plugin de automação do WhatsApp via ADB
└── Makefile                 # install, uninstall, lint, test, desktop, autostart
```

2. Evolução do Projeto

2.1 Antes do wdroid (estado inicial)

O fluxo manual para iniciar o WhatsApp via Waydroid envolvia os seguintes passos executados manualmente a cada sessão:

```
# Passo 1 – verificar se o container estava ativo
sudo systemctl start waydroid-container

# Passo 2 – iniciar sessão Android
waydroid session start &

# Passo 3 – esperar Android inicializar (sem feedback)
sleep 20

# Passo 4 – corrigir rede (quando necessário)
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -P FORWARD ACCEPT

# Passo 5 – abrir WhatsApp
waydroid app launch com.whatsapp
```

Problemas recorrentes: sem feedback de progresso, sem detecção de falhas, rede silenciosamente quebrada, múltiplas instâncias conflitantes, nenhum log persistente.

2.2 wdroid v1 (versão inicial — scripts simples)

A primeira iteração consistia em um único script bash com funções inline sem modularização. Características:

- Um único arquivo com ~200 linhas
- Sem state machine — sempre tentava iniciar tudo do zero
- Sem retry — falha imediata em qualquer etapa
- Sem logging em arquivo — apenas echo na tela
- Sem lockfile — execuções paralelas causavam corrupção de estado
- Sem sistema de backup

2.3 wdroid v2 (arquitetura atual)

A versão 2 foi uma reescrita completa motivada pelos bugs crônicos da v1. As principais mudanças arquiteturais foram:

Aspecto	v1	v2
Estrutura	Monolítico (1 arquivo)	Modular (core + modules + commands + plugins)
State machine	Inexistente	STOPPED → CONTAINER_ONLY → SESSION_RUNNING → APP_RUNNING
Retry	Nenhum	Backoff exponencial em todas as operações críticas
Logging	echo simples	Estruturado: timestamp, nível, arquivo diário rotativo
Lockfile	Inexistente	PID-based com detecção de processo órfão
Diagnóstico	Inexistente	doctor com health score (0–10) e --fix automático
Backup	Inexistente	Backup seguro com parada automática do container
Plugins	Inexistente	Sistema dinâmico via source + plugin_main()
Rede	Correção manual	Auto-detecção e correção via fix_network()
Testes/Lint	Inexistente	make lint (shellcheck) + make test (bash -n)

3. Arquitetura Técnica

3.1 Carregamento de Módulos

O wdroid usa carregamento on-demand via source para evitar carregar código desnecessário. O entrypoint (bin/wdroid) carrega apenas o core (config, logger, lock, utils, state). Os módulos são carregados pelos comandos que precisam deles:

```
# Dentro de commands/start.sh:
_load_modules container session network app

# Implementação em bin/wdroid:
_load_modules() {
    for mod in "$@"; do
        source "$BASE_DIR/modules/$mod.sh"
    done
}
```

Isso garante que um wdroid logs não carrega os módulos container, session ou network desnecessariamente.

3.2 State Machine

O coração do wdroid v2 é a state machine implementada em core/state.sh. Ela resolve o estado atual do ambiente inspecionando o sistema — não confiando em arquivos de estado que podem ficar desatualizados:

```
get_state() {
    # Nível 1: systemd diz se o container está ativo
    if ! systemctl is-active --quiet "waydroid-container"; then
        echo "STOPPED"; return
    fi
    # Nível 2: waydroid status diz se há sessão Android
    if ! waydroid status 2>/dev/null | grep -q "Session: RUNNING"; then
        echo "CONTAINER_ONLY"; return
    fi
    # Nível 3: pidof verifica se o app está em execução
    if waydroid shell pidof "com.whatsapp" &>/dev/null; then
        echo "APP_RUNNING"; return
    fi
    echo "SESSION_RUNNING"
}
```

A transição de estados no comando start é incremental — ele parte do estado atual e avança apenas o necessário:

```
case "$STATE" in
    STOPPED)      start_container; start_session ;;
    CONTAINER_ONLY) start_session ;;           # container já ativo
    SESSION_RUNNING|APP_RUNNING) : ;;         # nada a fazer
esac
launch_app "$APP_PACKAGE"
```

3.3 Sistema de Retry com Backoff

A função `retry` em `core/utls.sh` garante resiliência em operações de rede e inicialização que podem falhar transitatoriamente:

```
retry() {
    local n=0
    local cmd=("$@")
    until "${cmd[@]}"; do
        ((n++))
        if ((n >= RETRY_MAX)); then
            die "Falhou após $n tentativas: ${cmd[*]}"
        fi
        warn "Tentativa $n/$RETRY_MAX falhou. Aguardando ${RETRY_DELAY}s..."
        sleep "$RETRY_DELAY"
    done
}

# Uso: retry sudo systemctl start waydroid-container
# Configuração em core/config.sh:
# RETRY_MAX=5    RETRY_DELAY=1
```

3.4 Sistema de Logging

O logger (`core/logger.sh`) grava simultaneamente na tela com cores ANSI e em arquivo diário:

```
LOG_DIR="${WDROID_LOG_DIR:-$HOME/.wdroid/logs}"
_LOG_FILE="$LOG_DIR/wdroid-$(date +%Y%m%d).log"

# Formato no arquivo:
# [2026-04-09 06:52:16] [INFO ] Container já ativo.
# [2026-04-09 06:52:16] [WARN ] Rede com problema...

# Funções disponíveis:
# log "msg"      → [INFO]  verde no terminal + arquivo
# warn "msg"     → [WARN]  amarelo no terminal + arquivo
# error "msg"    → [ERROR] vermelho no stderr + arquivo
# die "msg"      → error + exit 1
# header "msg"   → banner == colorido + arquivo
# section "msg"  → título de seção [ msg ]
# ok "msg"       → ✓ verde
# fail "msg"     → ✗ vermelho
# notice "msg"   → ! amarelo
```

3.5 Lockfile Anti-concorrência

O `core/lock.sh` impede execuções paralelas que poderiam corromper o estado do Waydroid. O diferencial é a detecção de processos órfãos:

```
acquire_lock() {
    if [ -f "$LOCK_FILE" ]; then
        local pid
        pid=$(cat "$LOCK_FILE" 2>/dev/null)
        if kill -0 "$pid" 2>/dev/null; then
            # Processo ainda vivo → abortar
            die "Outra instância em execução (PID: $pid)"
        else
            # PID morto → lockfile órfão, remover e continuar
            warn "Lockfile órfão (PID: $pid). Removendo..."
            rm -f "$LOCK_FILE"
        fi
    fi
}
```

```

echo $$ > "$LOCK_FILE"
trap "_release_lock" EXIT INT TERM
}

# Comandos de leitura usam acquire_lock_soft:
# status, logs, doctor, help — não bloqueiam, apenas avisam

```

4. Módulos em Detalhe

4.1 modules/container.sh

Abstrai o controle do serviço systemd waydroid-container.service. Todas as operações usam retry para tolerar falhas transitórias do systemd:

Função	Descrição
start_container()	retry systemctl start + wait_for is-active (timeout: 10s)
stop_container()	systemctl stop (não usa retry — parada deve ser imediata)
restart_container()	stop → start com wait_for
is_container_running() ()	systemctl is-active --quiet (usado pela state machine)
enable_autostart()	systemctl enable
disable_autostart()	systemctl disable

4.2 modules/session.sh

Gerencia a sessão Android (processo waydroid session start). A inicialização é assíncrona — o módulo aguarda ativamente via wait_for:

```

start_session() {
    waydroid session start &          # processo em background
    wait_for \
        "waydroid status 2>/dev/null | grep -q 'Session: RUNNING'" \
        "$SESSION_TIMEOUT" \         # timeout: 15s (configurável)
        "sessão Android"
}

```

4.3 modules/network.sh

Diagnóstico e correção da rede do container Android. O Waydroid cria uma interface virtual (waydroid0) e precisa de IP forwarding + regras de iptables para ter acesso à internet:

Verificação	Comando interno
check_network()	ip addr show waydroid0
check_ip_forward()	cat /proc/sys/net/ipv4/ip_forward == "1"

check_default_route()	waydroid shell ip route grep "default"
fix_network()	restart_container + sysctl ip_forward + iptables FORWARD ACCEPT
network_health()	Retorna score 0–3 (1 ponto por verificação)

4.4 modules/app.sh

Interface com apps Android via waydroid app e ADB. O ADB conecta na porta 5555 (exposição local do Waydroid):

Função	Implementação
launch_app(pkg)	waydroid app launch <pacote>
install_apk(file)	waydroid app install <arquivo.apk>
list_apps()	waydroid app list
adb_connect()	adb connect localhost:5555
send_text(msg)	waydroid shell input text <msg>
capture_screen(f)	screencap /sdcard/ss.png + adb pull

4.5 modules/backup.sh

O backup_safe() para o container antes de copiar /var/lib/waydroid para garantir consistência (sem writes concorrentes do Android):

```
backup_safe() {
    local was_running=false
    if is_container_running; then
        was_running=true
        stop_session && stop_container # para tudo
        sleep 1
    fi
    sudo cp -r "$WAYDROID_DATA_DIR" "$dest/"
    if $was_running; then # restaura estado anterior
        start_container && sleep 2 && start_session
    fi
}
```

5. Referência Completa de Comandos

5.1 Ciclo de Vida

Comando	O que faz
wdroid start	Lê estado atual, avança apenas o necessário, lança WhatsApp
wdroid stop	Encerra sessão e container na ordem inversa

<code>wdroid restart</code>	<code>stop + sleep 1 + start</code>
<code>wdroid fix</code>	<code>fix_network + restart_container + start_session + doctor</code>
<code>wdroid reset</code>	Backup opcional → apaga <code>/var/lib/waydroid</code> → <code>waydroid init</code> novo

5.2 Observabilidade

Comando	O que faz
<code>wdroid status</code>	Estado, container, sessão, rede, KVM, Wayland, logs recentes
<code>wdroid doctor</code>	Health check de 10 pontos com score percentual
<code>wdroid doctor --fix</code>	Mesma coisa + executa fix automático para cada falha
<code>wdroid logs</code>	Últimas 50 linhas do <code>journalctl waydroid-container</code>
<code>wdroid logs wdroid</code>	Últimas 50 linhas do arquivo <code>~/.wdroid/logs/wdroid-YYYYMMDD.log</code>
<code>wdroid logs all</code>	container + wdroid juntos
<code>wdroid logs follow</code>	<code>journalctl -f</code> do container (tempo real)
<code>wdroid logs all 200</code>	Especifica número de linhas como 2º argumento

5.3 Apps e ADB

Comando	O que faz
<code>wdroid launch [pacote]</code>	Abre app (padrão: <code>com.whatsapp</code>)
<code>wdroid install-apk <f></code>	Instala APK via <code>waydroid app install</code>
<code>wdroid apps</code>	Lista apps instalados
<code>wdroid adb</code>	<code>adb connect localhost:5555</code>
<code>wdroid send-text <msg></code>	Injeta texto via <code>waydroid shell input text</code>
<code>wdroid screenshot [f]</code>	<code>screencap + adb pull</code> → arquivo PNG local
<code>wdroid multi-window</code>	<code>waydroid prop set persist.waydroid.multi_windows true</code>

5.4 Backup

Comando	O que faz
<code>wdroid backup create</code>	Cria backup timestampado em <code>~/.wdroid/backups/</code>
<code>wdroid backup restore</code>	Lista backups e restaura o selecionado (confirmação "yes")
<code>wdroid backup list</code>	Lista backups com data e tamanho
<code>wdroid backup clean</code>	Mantém apenas os N mais recentes (padrão: 3)

[N]	
-----	--

5.5 Plugins

Comando	O que faz
<code>wdroid plugin list</code>	Lista plugins com descrição extraída de # DESCRIPTION:
<code>wdroid plugin run <nome> [args]</code>	Carrega plugin via source e chama plugin_main()
<code>wdroid plugin install <arquivo></code>	Copia script para ~/.wdroid/plugins/
<code>wdroid plugin remove <nome></code>	Remove plugin do diretório

6. Dependências e Requisitos

6.1 Requisitos de Sistema

Requisito	Versão mínima	Verificação
Debian GNU/Linux	11 (Bullseye)+	<code>cat /etc/os-release</code>
Kernel Linux	5.15+	<code>uname -r</code>
Arquitetura	x86_64	<code>uname -m</code>
Sessão Wayland	Qualquer	<code>echo \$XDG_SESSION_TYPE</code>
KVM (recomendado)	Habilitado	<code>lsmod grep kvm</code>
Espaço em disco	3 GB+	<code>df -h /var/lib/waydroid</code>

6.2 Dependências de Runtime

Pacote	Versão no Trixie	Papel no wdroid
waydroid	1.6.2	Runtime principal — container Android
libgbinder	1.1.43	Comunicação com o binder do kernel
libglibutil	1.0.80	Utilidades GLib para libgbinder
python3-gbinder	1.3.1	Bindings Python do gbinder (usado pelo waydroid)

iptables	1.8.11-2	Regras de firewall para rede do container
adb	34.0.5-12	Android Debug Bridge (automação de apps)
curl	8.14.1	Download de imagens e APKs
ca-certificates	20250419	Validação TLS para downloads
systemd	built-in	Gerencia waydroid-container.service
bash	5.2+	Runtime dos scripts wdroid

6.3 Dependências de Desenvolvimento (opcionais)

Ferramenta	Uso
shellcheck	make lint — análise estática de todos os scripts bash
make	Automação: install, uninstall, test, lint, desktop, autostart

6.4 Módulos do Kernel Necessários

```
# Para processadores Intel:
sudo modprobe kvm
sudo modprobe kvm_intel

# Para processadores AMD:
sudo modprobe kvm
sudo modprobe kvm_amd

# Para persistir no boot:
echo "kvm_intel" | sudo tee /etc/modules-load.d/kvm.conf
# ou kvm_amd

# Verificação:
lsmod | grep kvm
ls -la /dev/kvm
```

6.5 Configurações de Sistema

IP Forwarding: Necessário para o container ter acesso à internet. O wdroid ativa automaticamente via sysctl, mas desaparece no reboot. Para persistir: `echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-waydroid.conf`

iptables vs nftables: Debian 13 usa iptables-nft por padrão. O wdroid chama iptables -P FORWARD ACCEPT — que funciona com iptables-nft, mas se houver conflito, mudar para iptables-legacy via: `sudo update-alternatives --set iptables /usr/sbin/iptables-legacy`

7. Guia de Instalação Completo

7.1 Instalação do Waydroid no Debian 13 Trixie

O repositório oficial do Waydroid suporta Trixie nativamente desde 2026. O script de bootstrap detecta o codename automaticamente:

```
# Passo 1 – Instalar dependências base
sudo apt update
sudo apt install -y curl ca-certificates iptables adb

# Passo 2 – Adicionar repositório Waydroid
curl https://repo.waydro.id | sudo bash
# Resultado: deb [signed-by=/usr/share/keyrings/waydroid.gpg]
#           https://repo.waydro.id/ trixie main

# Passo 3 – Instalar Waydroid e dependências
sudo apt install -y waydroid
# Instala: waydroid 1.6.2, libgbinder 1.1.43,
#          libglibutil 1.0.80, python3-gbinder 1.3.1

# Passo 4 – Inicializar imagem Android (LineageOS 20, ~1GB)
sudo waydroid init
# Baixa system.zip (~838 MB) e vendor.zip (~189 MB)
# de sourceforge.net/projects/waydroid/

# Passo 5 – Habilitar KVM (Intel)
sudo modprobe kvm kvm_intel
sudo usermod -aG kvm "$USER"

# Passo 6 – Criar grupo waydroid (não criado automaticamente no Trixie)
sudo groupadd waydroid
sudo usermod -aG waydroid "$USER"
```

7.2 Instalação do wdroid

```
# Extrair e instalar
tar -xzf wdroid-v2.tar.gz
cd wdroid-v2
make install

# make install executa:
# 1. chmod +x em todos os scripts (make permissions)
# 2. mkdir -p ~/.wdroid ~/.wdroid/plugins
# 3. cp -r bin core modules commands plugins Makefile README.md ~/.wdroid/
# 4. sudo ln -sf ~/.wdroid/bin/wdroid /usr/local/bin/wdroid

# Verificar instalação:
wdroid version  # → wdroid v2.0.0
wdroid help
```

7.3 Fix pós-instalação: bug ((TOTAL++)) no doctor.sh

Com set -euo pipefail ativo, incremento aritmético de variáveis zeradas causa exit silencioso. Aplicar o fix:

```
# Corrigir doctor.sh – troca ((VAR++)) por VAR=$((VAR + 1))
```

```
sed -i 's/((TOTAL++))/TOTAL=$((TOTAL + 1))/g; s/((SCORE++))/SCORE=$((SCORE + 1))/g' \
~/.wdroid/commands/doctor.sh

# Corrigir utils.sh – mesmo problema no retry()
sed -i 's/((n++))/n=$((n + 1))/g' ~/.wdroid/core/utils.sh

# Verificar:
wdroid doctor
```

7.4 Instalar WhatsApp

```
# 1. Iniciar container e sessão Android
wdroid start

# 2. Baixar APK oficial
curl -L "https://www.whatsapp.com/android/current/WhatsApp.apk" \
-o ~/WhatsApp.apk

# 3. Instalar
wdroid install-apk ~/WhatsApp.apk
# ou diretamente:
waydroid app install ~/WhatsApp.apk

# 4. Abrir
wdroid launch com.whatsapp
```

8. Bugs Encontrados e Resoluções

Esta seção documenta os bugs mais críticos encontrados durante o desenvolvimento e implantação do wdroid v2 no Debian 13 Trixie.

Bug #1: set -e aborta silenciosamente em ((VAR++))

Sintoma: wdroid doctor travava silenciosamente ao chegar em [Dependências], sem nenhuma mensagem de erro.

Causa raiz: Em bash com set -euo pipefail, a expressão aritmética ((0)) retorna exit code 1 (expressão falsa). Quando TOTAL e SCORE iniciam em 0, o primeiro ((TOTAL++)) retorna 1 e o shell aborta imediatamente.

Solução: Substituir ((TOTAL++)) por TOTAL=\$((TOTAL + 1)) e ((SCORE++)) por SCORE=\$((SCORE + 1)) — a forma aritmética de substituição nunca gera exit code de erro.

Bug #2: Grupo "waydroid" não existe no Trixie

Sintoma: "usermod: grupo waydroid não existe" ao executar sudo usermod -aG waydroid \$USER.

Causa raiz: O pacote waydroid 1.6.2 não cria o grupo waydroid durante a instalação no Debian 13. Versões anteriores criavam via postinst script que foi removido.

Solução: sudo groupadd waydroid && sudo usermod -aG waydroid "\$USER" && newgrp

Bug #3: Container mostra CONTAINER_ONLY mesmo após waydroid session start

Sintoma: wdroid start reportava "Container já ativo" mas não avançava para SESSION_RUNNING mesmo com o Android funcionando.

Causa raiz: O comando waydroid session start é assíncrono e retorna imediatamente antes do Android estar pronto. A mensagem "Session is already running" aparecia porque uma sessão estava sendo inicializada, mas waydroid status ainda reportava Session: STOPPED.

Solução: A função wait_for em utils.sh já resolve isso — aguarda até waydroid status | grep "Session: RUNNING" com timeout configurável (SESSION_TIMEOUT=15s). O problema era que o start_session não estava sendo chamado corretamente em determinados estados de race condition.

Bug #4: iptables-nft vs iptables-legacy — regras não persistiam

Sintoma: wdroid doctor --fix reportava "Rede corrigida" mas o WhatsApp continuava sem internet após alguns minutos.

Causa raiz: Debian 13 usa iptables-nft (wrapper do nftables). As regras adicionadas via iptables -P FORWARD ACCEPT funcionavam temporariamente mas eram sobrescritas pelo nftables ao reiniciar o Waydroid. Os dois sistemas de firewall coexistiam sem sincronização.

Solução: Selecionar iptables-legacy explicitamente: sudo update-alternatives --set iptables /usr/sbin/iptables-legacy. Alternativamente, adicionar as regras equivalentes diretamente no nftables.

Bug #5: Lockfile órfão após Ctrl+C durante wdroid start

Sintoma: Após interromper wdroid start com Ctrl+C, qualquer tentativa posterior retornava "Outra instância em execução (PID: XXXX)" mesmo com nenhum processo wdroid rodando.

Causa raiz: A trap EXIT INT TERM para _release_lock() funciona corretamente, mas Ctrl+C durante certos waits (ex: inside waydroid session start &) deixava o trap em estado inconsistente em algumas versões do bash 5.2.

Solução: O acquire_lock() já tem detecção de órfão via kill -0 \$pid: se o PID não existe mais, o lockfile é removido automaticamente. Para limpeza manual: rm /tmp/wdroid.lock ou make clean.

Bug #6: waydroid app install retornava sem output de confirmação

Sintoma: wdroid install-apk ~/WhatsApp.apk executava sem erros mas sem confirmar se o APK foi instalado. Não era possível saber se teve sucesso.

Causa raiz: O comando waydroid app install não produz saída visível em caso de sucesso — apenas em caso de erro. O run() wrapper do wdroid usa exec direto sem capturar stdout.

Solução: Verificar com wdroid apps | grep com.whatsapp após instalar, ou usar waydroid app install diretamente para ver output completo do processo de instalação.

Bug #7: APK do WhatsApp.com é ARM — pode não rodar nativamente em x86_64

Sintoma: WhatsApp abre mas trava ou fica em loop de inicialização na imagem VANILLA x86_64.

Causa raiz: O APK oficial de whatsapp.com é compilado para ARM (armeabi-v7a + arm64-v8a). A imagem VANILLA do Waydroid x86_64 não inclui tradução de binários ARM por padrão — ao contrário da imagem GAPPS que inclui libhoudini.

Solução: Usar a imagem GAPPS (sudo waydroid init -s GAPPS) que inclui libhoudini para tradução ARM → x86, ou baixar um APK com suporte x86_64 do APKMirror (filtrar por "x86_64" ou "x86").

9. Sistema de Plugins

O wdroid v2 implementa um sistema de plugins dinâmico baseado em source do bash. Qualquer arquivo .sh com a função plugin_main() pode ser carregado como plugin.

9.1 Estrutura de um Plugin

```
#!/bin/bash
# DESCRIPTION: Descrição curta (exibida em wdroid plugin list)

# O plugin tem acesso a todos os módulos e funções do core
# via _load_modules, pois é executado no mesmo processo bash

plugin_main() {
    local action="${1:-help}"
    shift || true

    case "$action" in
        minha-acao)
            _load_modules app session # carregar módulos necessários
            require_state SESSION_RUNNING
            launch_app "com.meuapp"
            ;;
        help|*)
            echo "Uso: wdroid plugin run meu-plugin minha-acao"
            ;;
    esac
}
```

9.2 Plugin whatsapp.sh (incluído)

O plugin whatsapp.sh demonstra o uso do sistema com ações práticas:

Ação	Implementação
open	require_state + launch_app com.whatsapp
send <texto>	require_state + send_text via waydroid shell input text

screenshot	screencap /sdcard/ss.png + adb pull
adb	adb connect localhost:5555

9.3 Como Instalar e Usar Plugins

```
# Instalar plugin externo
wdroid plugin install ~/meu-plugin.sh

# Listar plugins disponíveis
wdroid plugin list

# Executar plugin
wdroid plugin run whatsapp open
wdroid plugin run whatsapp send "Olá!"

# Remover plugin
wdroid plugin remove whatsapp
```

10. Configuração e Personalização

10.1 Variáveis de Ambiente

Variável	Padrão e descrição
WDROID_BACKUP_DIR	\$HOME/.wdroid/backups — diretório de backups
WDROID_LOG_DIR	\$HOME/.wdroid/logs — diretório de logs diários

10.2 Configurações em core/config.sh

```
WDROID_VERSION="2.0.0"

# Container
WAYDROID_CONTAINER="waydroid-container"    # nome do serviço systemd
WAYDROID_DATA_DIR="/var/lib/waydroid"      # dados do Android

# Rede
WAYDROID_IFACE="waydroid0"                 # interface virtual
WAYDROID_GATEWAY="192.168.240.1"           # gateway do container

# App padrão
APP_PACKAGE="com.whatsapp"                 # aberto pelo wdroid start

# Timeouts e retries
RETRY_MAX=5                               # máximo de tentativas em retry()
RETRY_DELAY=1                             # segundos entre tentativas
CONTAINER_TIMEOUT=10                       # timeout para container ficar ativo
SESSION_TIMEOUT=15                         # timeout para sessão Android iniciar
```

10.3 Integrações de Desktop

```
# Atalho no menu de aplicativos (cria .desktop no ~/.local/share/applications/)
make desktop

# Iniciar automaticamente ao logar no desktop
make autostart

# Remover autostart
make no-autostart
```

10.4 Linting e Testes

```
# Análise estática com shellcheck (instalar: sudo apt install shellcheck)
make lint

# Verificação de sintaxe bash em todos os scripts
make test

# Remover temporários (lockfile e state file)
make clean
```

11. Troubleshooting

Sintoma	Causa provável	Solução
wdroid doctor trava em [Dependências]	Bug ((VAR++)) com set -e	Aplicar fix do Bug #1 (seção 8)
"grupo waydroid não existe"	Pacote não cria grupo no Trixie	sudo groupadd waydroid
WhatsApp sem internet	IP forward / iptables	wdroid doctor --fix
Container travado	Estado inconsistente	wdroid restart
"Outra instância em execução"	Lockfile órfão	rm /tmp/wdroid.lock
WhatsApp fecha imediatamente	APK ARM em imagem VANILLA	Usar GAPPS ou APK x86_64
waydroid session start lento	KVM não ativo	modprobe kvm kvm_intel
Tela não aparece	Sessão X11 em vez de Wayland	Logar em sessão Wayland
wdroid status: Wayland não detectado	\$XDG_SESSION_TYPE não é wayland	Verificar sessão no display manager

11.1 Diagnóstico Rápido

```
# Checklist completo em ordem:

# 1. KVM ativo?
lsmod | grep kvm && echo "OK" || echo "FALTA: sudo modprobe kvm kvm_intel"
```



```
# 2. Wayland?
echo $XDG_SESSION_TYPE # deve ser "wayland"

# 3. Container?
systemctl is-active waydroid-container

# 4. Sessão?
waydroid status | grep Session

# 5. Rede?
ip addr show waydroid0
cat /proc/sys/net/ipv4/ip_forward # deve ser 1

# 6. Score completo:
wdroid doctor
```